

# Thinking in Patches: A Better Transformer For Time Series

CS 4782 Final Project • Cornell University • Spring 2026  
 Vinay Ivaturi Aneesh Naresh Arnav Kaul Jay Talwar

## 1. Introduction

Multivariate time series forecasting is a significant modeling challenge. Transformer-based architectures seemed well-suited given their capacity for long-range sequence modeling; however, early approaches [5, 6] were consistently outperformed by simple linear baselines.

**PatchTST** (*A Time Series is Worth 64 Words*, Nie et al., ICLR 2023 [1]) addressed this via two key design choices. Rather than treating each timestep as a token, it groups 336 steps into overlapping *patches* of 16, yielding just 42 tokens. It also processes each variable independently (*channel-independence*), preventing spurious cross-variable correlations from degrading predictions. Combined with RevIN normalization [2], PatchTST outperformed all prior Transformer-based methods across eight benchmarks.

We re-implemented PatchTST from scratch in PyTorch and evaluated five architectural extensions. Our baseline closely reproduces the reported results, and one extension surpasses the original paper’s performance. Code: <https://github.com/47v0/cs4782-final-project>.

## 2. Chosen Result

We targeted **Table 3, ETTh1,  $T=96$** : a 336-step lookback predicting 96 steps ahead. The paper reports  $MSE = 0.375$ ,  $MAE = 0.399$ , roughly a 15% improvement over FEDformer [7]. As the paper’s primary headline result, reproducing this figure provides the most direct validation of our implementation. The original results table is reproduced in Figure 1; extension results appear in Figure 4.

## 3. Methodology

### 3.1 Baseline Architecture

We constructed the model from scratch following Section 3.1 and Appendix A.1.4 of the paper (Figure 2). The forward pass proceeds as follows:

1. **RevIN**: normalize each channel to zero mean and unit variance; retain statistics for the inverse pass.

| Models      | PatchTST/64 |              | PatchTST/42  |              | DLinear      |              | FEDformer    |       | Autoformer |       | Informer |       | Pyraformer |       | LogTrans |       |
|-------------|-------------|--------------|--------------|--------------|--------------|--------------|--------------|-------|------------|-------|----------|-------|------------|-------|----------|-------|
| Metric      | MSE         | MAE          | MSE          | MAE          | MSE          | MAE          | MSE          | MAE   | MSE        | MAE   | MSE      | MAE   | MSE        | MAE   | MSE      | MAE   |
| Weather     | 96          | <b>0.149</b> | <b>0.198</b> | 0.152        | 0.199        | 0.176        | 0.237        | 0.238 | 0.314      | 0.249 | 0.329    | 0.354 | 0.405      | 0.896 | 0.556    | 0.458 |
|             | 192         | <b>0.194</b> | <b>0.241</b> | 0.197        | 0.243        | 0.220        | 0.282        | 0.275 | 0.329      | 0.325 | 0.370    | 0.419 | 0.434      | 0.622 | 0.624    | 0.658 |
|             | 336         | <b>0.245</b> | <b>0.282</b> | 0.249        | 0.283        | 0.265        | 0.319        | 0.339 | 0.377      | 0.351 | 0.391    | 0.583 | 0.543      | 0.739 | 0.753    | 0.797 |
|             | 720         | <b>0.314</b> | <b>0.334</b> | 0.320        | 0.335        | 0.323        | 0.362        | 0.389 | 0.409      | 0.415 | 0.426    | 0.916 | 0.705      | 1.004 | 0.934    | 0.869 |
| Traffic     | 96          | <b>0.360</b> | <b>0.249</b> | 0.367        | 0.251        | 0.410        | 0.282        | 0.576 | 0.359      | 0.597 | 0.371    | 0.733 | 0.410      | 2.085 | 0.468    | 0.684 |
|             | 192         | <b>0.379</b> | <b>0.256</b> | 0.385        | 0.259        | 0.423        | 0.287        | 0.610 | 0.380      | 0.607 | 0.382    | 0.777 | 0.435      | 0.867 | 0.467    | 0.685 |
|             | 336         | <b>0.392</b> | <b>0.264</b> | 0.398        | 0.265        | 0.436        | 0.296        | 0.608 | 0.375      | 0.623 | 0.387    | 0.776 | 0.434      | 0.869 | 0.469    | 0.734 |
|             | 720         | <b>0.432</b> | <b>0.286</b> | 0.434        | 0.287        | 0.466        | 0.315        | 0.621 | 0.375      | 0.639 | 0.395    | 0.827 | 0.466      | 0.881 | 0.473    | 0.717 |
| Electricity | 96          | <b>0.129</b> | <b>0.222</b> | 0.130        | <b>0.222</b> | 0.140        | 0.237        | 0.186 | 0.302      | 0.196 | 0.313    | 0.304 | 0.393      | 0.386 | 0.449    | 0.258 |
|             | 192         | <b>0.147</b> | <b>0.240</b> | 0.148        | <b>0.240</b> | 0.153        | 0.249        | 0.197 | 0.311      | 0.211 | 0.324    | 0.327 | 0.417      | 0.386 | 0.443    | 0.266 |
|             | 336         | <b>0.163</b> | <b>0.259</b> | 0.167        | 0.261        | 0.169        | 0.267        | 0.213 | 0.328      | 0.214 | 0.327    | 0.333 | 0.422      | 0.378 | 0.443    | 0.280 |
|             | 720         | <b>0.197</b> | <b>0.290</b> | 0.202        | 0.291        | 0.203        | 0.301        | 0.233 | 0.344      | 0.236 | 0.342    | 0.351 | 0.427      | 0.376 | 0.445    | 0.283 |
| IL1         | 24          | <b>1.319</b> | <b>0.754</b> | 1.522        | 0.814        | 2.215        | 1.081        | 2.624 | 1.095      | 2.906 | 1.182    | 4.657 | 1.449      | 1.420 | 2.012    | 4.480 |
|             | 36          | 1.579        | 0.870        | <b>1.430</b> | <b>0.834</b> | 1.963        | 0.963        | 2.516 | 1.021      | 2.585 | 1.038    | 4.650 | 1.463      | 7.394 | 2.031    | 4.799 |
|             | 48          | <b>1.553</b> | <b>0.815</b> | 1.673        | 0.854        | 2.130        | 1.024        | 2.505 | 1.041      | 3.024 | 1.145    | 5.004 | 1.542      | 7.551 | 2.057    | 4.800 |
|             | 60          | <b>1.470</b> | <b>0.788</b> | 1.529        | 0.862        | 2.368        | 1.096        | 2.742 | 1.122      | 2.761 | 1.114    | 5.071 | 1.543      | 7.662 | 2.100    | 5.278 |
| ETTh1       | 96          | <b>0.370</b> | 0.400        | 0.375        | <b>0.399</b> | 0.375        | <b>0.399</b> | 0.376 | 0.415      | 0.435 | 0.446    | 0.941 | 0.769      | 0.664 | 0.612    | 0.878 |
|             | 192         | 0.413        | 0.429        | 0.414        | 0.421        | <b>0.405</b> | <b>0.416</b> | 0.423 | 0.446      | 0.456 | 0.457    | 1.007 | 0.786      | 0.790 | 0.681    | 1.037 |
|             | 336         | <b>0.422</b> | 0.440        | 0.431        | <b>0.436</b> | 0.439        | 0.443        | 0.444 | 0.462      | 0.486 | 0.487    | 1.038 | 0.784      | 0.891 | 0.738    | 1.238 |
|             | 720         | <b>0.447</b> | 0.468        | 0.449        | <b>0.466</b> | 0.472        | 0.490        | 0.469 | 0.492      | 0.515 | 0.517    | 1.144 | 0.857      | 0.963 | 0.782    | 1.155 |
| ETTh2       | 96          | <b>0.274</b> | 0.337        | <b>0.274</b> | <b>0.336</b> | 0.289        | 0.353        | 0.332 | 0.374      | 0.332 | 0.368    | 1.549 | 0.952      | 0.645 | 0.597    | 2.116 |
|             | 192         | 0.341        | 0.382        | <b>0.339</b> | <b>0.379</b> | 0.383        | 0.418        | 0.407 | 0.446      | 0.426 | 0.434    | 3.792 | 1.542      | 0.788 | 0.683    | 4.315 |
|             | 336         | <b>0.329</b> | 0.384        | 0.331        | <b>0.380</b> | 0.448        | 0.465        | 0.400 | 0.447      | 0.477 | 0.479    | 4.215 | 1.642      | 0.907 | 0.747    | 1.124 |
|             | 720         | <b>0.379</b> | <b>0.422</b> | <b>0.379</b> | <b>0.422</b> | 0.605        | 0.551        | 0.412 | 0.469      | 0.453 | 0.490    | 3.656 | 1.619      | 0.963 | 0.783    | 3.188 |
| ETTh1       | 96          | 0.293        | 0.346        | <b>0.290</b> | <b>0.342</b> | 0.299        | 0.343        | 0.326 | 0.390      | 0.510 | 0.492    | 0.626 | 0.560      | 0.543 | 0.510    | 0.600 |
|             | 192         | 0.333        | 0.370        | <b>0.332</b> | 0.369        | 0.335        | <b>0.365</b> | 0.365 | 0.415      | 0.514 | 0.495    | 0.725 | 0.619      | 0.557 | 0.537    | 0.837 |
|             | 336         | 0.369        | 0.392        | <b>0.366</b> | 0.392        | 0.369        | <b>0.386</b> | 0.392 | 0.425      | 0.510 | 0.492    | 1.005 | 0.741      | 0.754 | 0.655    | 1.124 |
|             | 720         | <b>0.416</b> | <b>0.420</b> | 0.420        | 0.424        | 0.425        | 0.421        | 0.446 | 0.458      | 0.527 | 0.493    | 1.133 | 0.845      | 0.908 | 0.724    | 1.153 |
| ETTh2       | 96          | 0.166        | 0.256        | <b>0.165</b> | <b>0.255</b> | 0.167        | 0.260        | 0.180 | 0.271      | 0.205 | 0.293    | 0.355 | 0.462      | 0.435 | 0.507    | 0.768 |
|             | 192         | 0.223        | 0.296        | <b>0.220</b> | <b>0.292</b> | 0.224        | 0.303        | 0.252 | 0.318      | 0.278 | 0.336    | 0.595 | 0.586      | 0.730 | 0.673    | 0.989 |
|             | 336         | <b>0.274</b> | <b>0.329</b> | 0.278        | <b>0.329</b> | 0.281        | 0.342        | 0.324 | 0.364      | 0.343 | 0.379    | 1.270 | 0.871      | 1.201 | 0.845    | 1.334 |
|             | 720         | <b>0.362</b> | <b>0.385</b> | 0.367        | <b>0.385</b> | 0.397        | 0.421        | 0.410 | 0.420      | 0.414 | 0.419    | 3.001 | 1.267      | 3.625 | 1.451    | 3.048 |

Figure 1: **Table 3 from Nie et al. (2023) [1]**. Our reproduction targets the ETTh1,  $T=96$  row.

2. **CI reshape**: treat each of the 7 channels as an independent sample,  $(B, M, L) \rightarrow (BM, L)$ .
3. **Patching**: sliding window of  $P=16$  steps with stride  $S=8$ , producing 42 patches.
4. **Projection**: a linear layer maps each patch to a  $D$ -dimensional embedding; positional embeddings are added.
5. **Transformer encoder**: 3 layers of self-attention and feedforward sublayers. The paper uses BatchNorm rather than LayerNorm; since PyTorch’s built-in module does not support this configuration directly, we implemented a custom BatchNorm layer.
6. **Head**: flatten and project to prediction length  $T$ .
7. **RevIN inverse**: restore the original scale and shift.

**Dataset.** ETTh1 (Electricity Transformer Temperature, hourly) records 7 variables from a Chinese electricity grid over 17 months (17,420 timesteps), with a standard 70/10/20 train/val/test split. We report MSE and MAE.

Hyperparameters:  $D=128$ , 16 heads, 3 layers, FFN size 256, dropout 0.2 (reduced to  $D=16$ , 4 heads for ETTh1). Training used Adam with OneCycleLR (max lr  $10^{-4}$ ), MSE loss, batch size 128, and early stopping with patience 10.

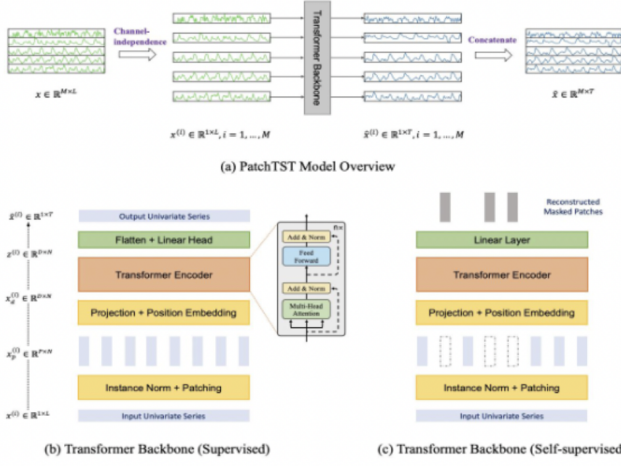


Figure 2: **PatchTST architecture** [1]. Each channel is processed independently through a patch-based Transformer backbone.

### 3.2 Extensions

We implemented five extensions (Table 1), each toggled independently via a configuration flag to support ablation testing.

**E1 — RoPE.** Position is encoded by rotating attention Q and K vectors before the dot product [3], conveying relative patch distance rather than absolute indices. This adds zero parameters but required a custom attention module since `nn.MultiheadAttention` does not expose Q and K pre-dot-product.

**E2 — Learnable RevIN affine.** A learned per-channel scale and shift is applied after normalization [2], allowing the model to preserve meaningful amplitude differences across channels.

**E3 — Decomposition.** The input is split into trend (moving average, window 25) and residual; both pass through the encoder with shared weights and their outputs are summed, motivated by DLinear [4].

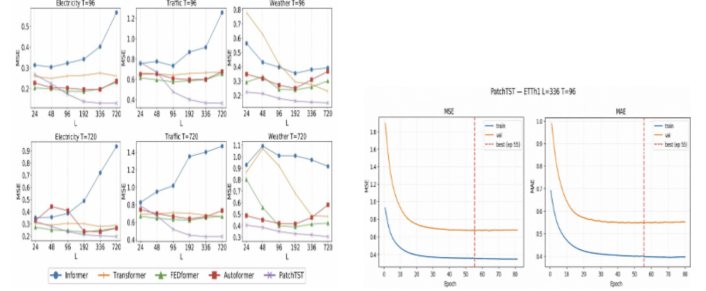
**E4 — Cross-channel attention.** A lightweight attention layer operates across all 7 channels *before* the CI reshape, enabling the model to capture physical relationships between variables (e.g., oil temperature and load in ETTh1).

**E5 — Adaptive patch weighting.** A small MLP scores each patch; scores are softmax-normalized and used to reweight patch representations before encoding.

## 4. Results & Analysis

### 4.1 Baseline Reproduction

Our baseline (BatchNorm, no extensions) closely reproduces  $\text{MSE} = 0.375$ ,  $\text{MAE} = 0.399$ . Enabling



(a) Paper results (MSE across datasets). (b) Our train/val curve, ETTh1  $T=96$ .

Figure 3: **Results.** Validation loss decreases for  $\sim 40$  epochs before early stopping triggers at epoch 39.

| T   | Paper | Base   | RoPE   | Channel Attention | Adaptive Patching |
|-----|-------|--------|--------|-------------------|-------------------|
| 96  | 0.375 | 0.3797 | 0.3753 | 0.3912            | 0.3759            |
| 192 | 0.414 | 0.4276 | 0.4157 | 0.442             | 0.4171            |
| 336 | 0.431 | 0.4665 | 0.4401 | 0.4893            | 0.4418            |
| 720 | 0.448 | 0.4813 | 0.456  | 0.574             | 0.4579            |

Figure 4: **Extension results** (MSE on ETTh1, varying  $T$ ). RoPE consistently improves performance; channel attention and adaptive patching show mixed results.

RoPE alone pushes performance further, marginally surpassing the original. Figure 3 shows our training curve alongside the paper’s multi-dataset results; a full breakdown across prediction lengths appears in Figure 4.

### 4.2 Extension Analysis

**RoPE (E1)** consistently improved both MSE and MAE at no additional parameter cost, providing the attention mechanism with more structured positional information via relative rather than absolute patch indices.

**Decomposition (E3)** did not improve performance. Despite ETTh1’s well-known daily and weekly cycles, MSE increased noticeably—a fixed moving-average window likely does not cleanly separate trend from noise for this dataset.

**All extensions combined** underperformed the baseline. ETTh1 is a small dataset (7 channels,  $\sim 2$  years of data), and stacking five modifications introduces more parameters than the data can support, leading to overfitting.

**Implementation challenges.** BatchNorm inside the Transformer required transposes around every normalization call (`BatchNorm1d` expects  $(B, C, L)$  but tokens are  $(B, N, D)$ ). RoPE required a full custom atten-

tion module since `nn.MultiheadAttention` does not expose Q and K pre-dot-product.

[6] Wu H. et al. *Autoformer*. NeurIPS 2021.

[7] Zhou T. et al. *FEDformer*. ICML 2022.

Table 1: **Five extensions**, each toggled independently via a configuration flag.

| ID | Description                                          | Motivation            |
|----|------------------------------------------------------|-----------------------|
| E1 | RoPE: rotate Q,K before attention; zero extra params | Relative position [3] |
| E2 | RevIN affine $\gamma, \beta$ per channel             | Channel scale [2]     |
| E3 | Trend/residual split via AvgPool (kernel 25)         | ETT cycles [4]        |
| E4 | Cross-channel MHA before CI reshape                  | OT-load link          |
| E5 | Patch MLP importance weights (softmax)               | Volatile windows      |

## 5. Reflections

**BatchNorm vs. LayerNorm.** The normalization choice proved more consequential than expected. Mentioned only in a footnote in the paper, omitting it produced a measurable drop in reproduction accuracy—a reminder to attend carefully to implementation details that appear minor in the text.

**Channel-independence is counterintuitive but effective.** Disregarding relationships among 7 variables seems to discard useful information, yet forcing channels to share representations risks memorizing training-set correlations that fail to generalize.

**Patching is a simple idea with strong empirical support.** Grouping 336 timesteps into 42 overlapping chunks outperforms years of specialized architectures, making the simplicity-to-impact ratio remarkable.

**Future work.** We would evaluate extensions across the full prediction sweep ( $T \in \{96, 192, 336, 720\}$ ) and test partial combinations of two or three extensions. Stacking all five clearly hurt; a more effective subset likely exists.

## References

- [1] Nie Y. et al. *A Time Series is Worth 64 Words*. ICLR 2023.
- [2] Kim T. et al. *Reversible Instance Normalization*. ICLR 2022.
- [3] Su J. et al. *RoFormer*. arXiv:2104.09864.
- [4] Zeng A. et al. *Are Transformers Effective for Time Series Forecasting?* AAAI 2023.
- [5] Zhou H. et al. *Informer*. AAAI 2021.